Contents

Delivery Plan — music-room **1**
0. The governing rule . . . 1
Phase 0 — Verify the requirements . . . 2
Phase 1 — Scope decision (issue O1) . . . 2
Phase 2 — Close the blocking decisions . . . 3
Phase 3 — Fill the design gaps . . . 4
Phase 4 — Build in vertical slices . . . 4
Phase 5 — Do these first, not last . . . 7
Phase 6 — Defence material . . . 7
Work split . . . 8
Risk register . . . 8
Progress tracking . . . 9
Related . . . 10

Delivery Plan — music-room

How to get from the subject PDF to a defensible submission.

Status: pre-implementation. `MusicRoom/` is empty. Every decision recorded so far is in [[decisions-index]]; everything still open is in [[sdd]] §17.

This plan sequences work. It does not decide anything — decisions go in ADRs, rules go in [[technical-guide-index]], design goes in [[sdd]].

Unknowns that shape the plan

Input	Value
Team size	**2** — one backend developer, one UI developer
Backend owner	one person; this plan is that person's plan
Deadline	`UNKNOWN - requires decision`
Services in scope	**all three** — [[ADR-0010-service-scope]]

No dates appear below. Phases are ordered by dependency, not by calendar. Put dates on them once the deadline is known.

Phases 1, 2 and 3 are complete. Seven decisions closed on 2026-09-24 (ADR-0010 through ADR-0016, see [[decisions-index]] and the Closed table in [[sdd]] §17), and the endpoint inventory is written: [[endpoint-inventory]]. Work starts at **Phase 4, slice S0**.

---

0. The governing rule

> Every sentence in the subject that says **"justify your choice"** becomes exactly one ADR. Everything else in the subject is a requirement, not a decision.

Subject	What it asks justified	Record	State
IV.1	backend technology and strategy	[[ADR-0001-backend-dotnet-narchitecture]]	done
III	the SDK must not do your work	[[ADR-0003-music-provider-youtube]]	done

Subject	What it asks justified	Record	State
V.4	REST as the API style	[[ADR-0005-api-style-rest-json]]	done
V.4	JSON as the exchange format	[[ADR-0005-api-style-rest-json]]	done
V.7	the concurrent-user figure	`docs/load/REPORT.md` — measurement, not an ADR	not started
V.6	password storage and federated login	[[ADR-0007-password-hashing-bcrypt]], [[ADR-0008-oidc-social-login]]	done
V.6, V.8	security conformance and code quality gates	[[ADR-0009-code-quality-gates]]	done

Requirements live in [[subject-requirements]]. Do not re-argue them; satisfy them.

Phase 0 — Verify the requirements

Goal: the distilled requirement list matches the PDF.

[[subject-requirements]] was distilled from `docs/subject/en.subject tr.pdf` once. At defence the **PDF** is authoritative, not the distillation.

Step	Output
Read the PDF with a teammate, line by line, against [[subject-requirements]]	corrections, if any
Fix any divergence in the Brain note	a verified requirement list

Exit condition: two people have read the PDF and agree the note is faithful.

Do this before anything else. Every later phase inherits this list.

Phase 1 — Scope decision (issue O1)

Goal: decide which 2 of the 3 services ship. This blocks the endpoint inventory, the schema, and the work split. Nothing meaningful can be planned before it.

Subject V.2 requires at least two of: Music Track Vote, Music Playlist Editor, Music Control Delegation.

Service	Relative difficulty	Where the grading risk sits
Music Playlist Editor	medium	real-time reorder race conditions (V.2.3)
Music Track Vote	**high**	vote race conditions **plus** the location+time licence mode (V.2.1)
Music Control Delegation	low	per-device licence state machine (V.2.2)

Notes for the decision, not a recommendation to adopt blindly:

- Track Vote carries the location+time licence requirement, which pulls in geofencing (issue O7). It is the only requirement currently marked `partial` in [[sdd]] §16.
- Editor and Delegation together still exercise realtime, concurrency, visibility and licence — the four things the subject grades hardest.
- Identity, Profile, Catalog and Audit are required whatever you choose. They are not optional scope.

Closed: [[ADR-0010-service-scope]] — all three services ship.

Consequence: the location+time licence mode became mandatory and is decided in [[ADR-0014-location-time-licence]]. The fallback if the schedule slips is to drop Control Delegation first; two services still satisfy V.2.

Prompt: §7 in [[prompt-library]].

Phase 2 — Close the blocking decisions

Ordered by what each one unblocks. Each is an ADR.

All closed on 2026-09-24.

Issue	Decision	ADR
O2	layout mirrors nArchitecture: `src/corePackages` + `src/MusicRoom` + `tests`	[[ADR-0011-solution-layout]]
O6	hosted transactional mail provider, free tier, behind `IMailService`	[[ADR-0013-mail-provider]]
O4	local Docker on a declared machine; GitHub Actions `dev`/`prod`, no deploy	[[ADR-0015-hosting-and-deployment]]
O5	Supabase Free tier, Frankfurt `eu-central-1`	[[ADR-0016-supabase-plan-region]]
O3	xUnit + NSubstitute + Testcontainers	[[ADR-0012-test-stack]]
O7	geofence **plus** optional event code, both evaluated server-side	[[ADR-0014-location-time-licence]]

Still open, on purpose: O8 SignalR backplane (not needed for a single-instance defence), O9 concrete schema (the first migration resolves it), O11 remote deployment target (opened by [[ADR-0015-hosting-and-deployment]] — Fly.io, Railway, Render or Azure Container Apps; **not Vercel**, which cannot host a long-running ASP.NET Core process or a persistent WebSocket).

Closing an issue — the full loop

```
1. Decide it yourself.                          (subject section II)
2. Write brain/decisions/ADR-XXXX-<slug>.md  Context / Decision / Consequences / Alternatives rejected
3. Add the row to brain/decisions/decisions-index.md
4. Remove the matching UNKNOWN in brain/
5. Close the row in docs/SDD.md §17 and fill the affected viewpoint section
6. Bump any standard the decision changes, regenerate its PDF
7. ./docs/build-pdf.sh
```

Skipping steps 4 to 6 is how a Brain rots into fiction. Prompt §10b runs the propagation pass.

Phase 3 — Fill the design gaps

With Phases 1 and 2 closed, these SDD gaps can be filled. Do them **before** writing code.

[[sdd]] section	Gap	Status
§5.1	project and folder layout	filled — [[ADR-0011-solution-layout]]
§5.2	which services	filled — all three
§9.1	**endpoint inventory**	**this phase.** Derived from the three service contexts plus Identity, Profile, Catalog, Audit
§11.2	device control state names	settled when the Delegation slice is built
§8.5	table and column definitions	the first migration — **not** this phase

Done. [[endpoint-inventory]] is written and is the contract the UI developer builds against. Remaining gaps there (E1-E4) are resolved by code, not by more design.

The endpoint inventory is written on paper first. Subject V.4 grades the API reference documentation; OpenAPI is generated from code, but the contract is designed before the code.

Prompt: §3 in [[prompt-library]].

Exit condition: every in-scope endpoint has a method, path, request, response, every status code, and its authorisation rule — both gates from [[sdd]] §13.2.

Phase 4 — Build in vertical slices

Never plan by layer. "Domain first, then Application" never produces something that runs. Each slice below cuts through all five projects and ends with something demonstrable.

#	Slice	Delivers	Subject
S0	Walking skeleton	solution per [[ADR-0011-solution-layout]], `Directory.Packages.props` with the set in [[ADR-0021-backend-dependency-set]], `docker compose` with a **local PostgreSQL container** ([[ADR-0026-environment-bring-up]]), `Makefile`, CI, `/health`, `/health/ready`, published OpenAPI	IV.1, V.8
S1	Identity	register (BCrypt cost >= 12), email verification, login, JWT + refresh rotation, password reset	V.1, V.6

#	Slice	Delivers	Subject
S1a	Mail provider adapter	**precondition: the T-25 rate limiter must exist first.** Owner decision 2026-09-27: an unauthenticated loop against `/auth/verification-requests` costs a log line today and burns the provider quota (T-19) the moment a real sender lands	
S1a	Mail provider adapter	the real hosted-provider implementation of `IMailService`, once the account and API key exist — issue O17	V.1
S1b	Secret vault	self-hosted Infisical under a `vault` compose profile, plus `InfisicalSecretProvider` behind the abstraction — issue O18, [[ADR-0029-containerised-dev-and-secret-vault]]	beyond V.8
S2	Audit + client context	required headers, middleware, audit behaviour, **SignalR hub filter**, audit table	V.6
S3	Profile	three visibility tiers, per-field server-side filtering, music preferences	V.1
S4	Catalog	YouTube adapter, `TrackRef`, caching, quota discipline	III
S5	Playlist Editor	REST + `PlaylistHub` + sparse ordering + concurrency token + parallel reorder tests	V.2.3
S6	Track Vote	REST + `EventHub` + unique-index vote arbitration + licence policy incl. geofence and event code + parallel vote tests	V.2.1
S7	Control Delegation + Playback	`PlaybackSession`, queue advance, device state machine, `ControlHub`, per-device licence	V.2.1, V.2.2

#	Slice	Delivers	Subject
S8	Social login	Google and Facebook via **OIDC**, PKCE, server-side ID token validation, account linking	V.1, V.5
S9	Capacity + conformance evidence	Gatling simulations under `tests/load/`, `docs/load/REPORT.md`; fill the Verified column in [[threat-register]] and the evidence in [[asvs-conformance]]	V.7, V.6

Why this order

- **S0 first.** Subject IV.1 requires a fresh clone to install its dependencies automatically. Build that while there is nothing to break. See Phase 5.
- **Verify the MediatR licence during S0**, before the package is added — issue O15. If the terms do not fit, the fallback is a hand-written dispatcher, and that is far cheaper to discover on day one than in slice S5.
- **S0 to S3 run on a local PostgreSQL container; Supabase joins at S4** ([[ADR-0026-environment-bring-up]]). Same engine, same migrations, different connection string.
- **External credentials are created just in time**: YouTube and the Supabase project before S4, Google and Facebook before S8. **Mail moved to the end of S1** (owner decision, 2026-09-24): SPEC-003 ships a Development-only logging `IMailService` so the verification flow is testable immediately, and the provider adapter lands as its own small spec once the key exists. V.1 still requires real delivery before the defence — the logging implementation is a staging post.
- **S2 early.** The audit log touches every mutating endpoint. Added late, it means revisiting every controller and every handler. Added at slice 2, every later slice inherits it.
- **S4 before S5.** Both services need track metadata. Building the adapter twice is waste.
- **S5 before S6.** Playlist Editor is the simpler of the two concurrency services. Solve optimistic concurrency and post-commit broadcast once, then reuse the pattern in Track Vote.
- **S6 carries the geofence.** Track Vote is the largest slice: vote races, three licence modes, and presence proof ([[ADR-0014-location-time-licence]]). Budget accordingly.
- **S7 is the drop candidate, with a caveat.** Dropping Control Delegation leaves two services and satisfies V.2 — but `PlaybackSession` also carries queue advance for Track Vote ([[ADR-0017-playback-session]]). If S7 is dropped, the session and its advance endpoint still ship; only delegation goes.
- **S8 after S1.** Account linking requires an existing local identity to link to.
- **S9 last but not final-day.** It depends on a working system, but it is graded material and cannot be improvised.

Work inside a slice is carried by a **spec**: a disposable work order under `docs/specs/`, which is also the unit an agent can execute unattended. See [[specs]] and [[prompt-library]] Part 2.

Unattended runs commit **one branch and one commit per spec, and never push** ([[ADR-0025-agent-git-protocol]]). A run refuses to start on a dirty working tree.

Definition of done — applies to every slice

A slice is not done until all of these are true:

- ☐ code under `MusicRoom/`, following the standards that apply
- ☐ builds clean with `TreatWarningsAsErrors` — no method over 40 lines, complexity over 10, or type over 15 methods; every `#pragma warning disable` justified

- ☐ [[owasp-top10-checklist]] and [[owasp-api-top10-checklist]] walked, with evidence per row
- ☐ every unfixed finding recorded in [[threat-register]] before merge
- ☐ the layer tests required by [[testing]] for the areas touched
- ☐ if it touched a concurrency hotspot: a **parallel** test asserting final database state
- ☐ negative authorisation cases tested — both gates, REST **and** hub
- ☐ OpenAPI regenerated, `docs/api/openapi.json` updated in the same PR
- ☐ migration recorded in [[migrations-index]]
- ☐ [[sdd]] §16 status updated honestly
- ☐ CI green

A sequential test proves nothing about a race. That is the one check reviewers most often accept by mistake.

Phase 5 — Do these first, not last

Both are graded, both are cheap at the start and expensive at the end.

Reproducible clone (subject IV.1, issue O10)

Target, working from slice S0 onward:

```
git clone  ->  cp .env.example .env  ->  fill values  ->  make up
```

Any additional manual step is a defect. Rules: [[ci-cd-containerization]].

Continuous integration (subject V.8)

Stand it up before the first real pull request. Gate: `dotnet restore`, `dotnet format --verify-no-changes`, `dotnet build -warnaserror`, `dotnet test`, `dotnet list package --vulnerable --include-transitive`, OpenAPI generation.

A red pipeline blocks merge. Disabling a check to land a branch is not an option.

Phase 6 — Defence material

Not code, but the grade depends on it.

Artifact	Requirement	What makes it valid
[[asvs-conformance]]	V.6	a stated scope with reasons, and evidence per met requirement — not a claim of full L2
`docs/load/REPORT.md`	V.7	a declared machine specification, a reproducible committed command, percentiles per concurrency step, REST and SignalR measured **separately**, the bottleneck named with evidence
[[threat-register]]	V.6	threat, mitigation, residual risk, accepted by. A written accepted risk is a defensible decision; an unwritten one is an oversight

Artifact	Requirement	What makes it valid
[[sdd]] §16	all	every mandatory requirement mapped to the design element that satisfies it, with an honest status
[[decisions-index]]	IV.1, III, V.4	every choice with its rejected alternatives — evaluators ask "why not X"
`docs/api/openapi.json`	V.4	readable without running the service

Bonus

Do not start it. Subject VI: the bonus is evaluated only if the mandatory part is flawless — complete and working without failure. Partial mandatory work plus bonus scores worse than mandatory work alone.

Work split

Subject III explicitly asks for tasks to be divided among team members.

The team is two people: one backend, one UI. The split is therefore across the client/server boundary, not across backend layers.

Owner	Scope
Backend developer	everything in this plan: S0 through S9, all under `MusicRoom/`
UI developer	the React Native client under `MusicRoomApp/` — same repository, separate root ([[ADR-0020-monorepo-layout]])
Shared	the API contract (the endpoint inventory from Phase 3), and the realtime event contracts in [[sdd]] §9.2

Consequences of a two-person split with a single backend owner:

- **The contract is the interface between the two people.** Phase 3's endpoint inventory is not paperwork; it is what unblocks the UI developer. Write it before S1.
- **Publish the OpenAPI document early and keep it current.** The UI developer consumes `docs/api/openapi.json`; a stale contract blocks them silently.
- **There is no second pair of backend eyes.** The concurrency slices (S5, S6, S7) are the highest-risk graded area and normally want pairing. Compensate with the parallel tests in the definition of done and a deliberate pass of prompt §4 on each one — a test that runs the operation concurrently is the substitute for a reviewer.
- **Both people must be able to defend the whole project**, not just their half. Subject section II is explicit about intellectual ownership. Walk the other person through [[sdd]] §16 before the defence.

Risk register

Risk	Impact	Mitigation
OWASP conformance treated as a review opinion	V.6 unmet, findings lost	the checklists need evidence per row; unfixed findings become [[threat-register]] rows before merge
Size limits disabled to land a branch	the gate becomes decorative	severity downgrades in `.editorconfig` require an ADR; `#pragma` escapes are visible and countable
BCrypt 72-byte truncation	two long passwords hash identically	64-character and 72-byte validation, enforced in the password validator
Audit log added late	revisit every endpoint	slice S2, early
Concurrency tested sequentially	a graded requirement silently unmet	parallel test in the definition of done
Load report left to the last day	V.7 unmet — an unmeasured number is worth nothing	S8, and the machine decided in Phase 2
Credential committed	**automatic project failure** (V.8)	`.env` git-ignored from commit 1; rotate-first leak procedure in [[secrets-management]]
Bonus started before mandatory is flawless	bonus not evaluated, mandatory weakened	Phase 6
Three services half-finished instead of two complete	subject VI: mandatory must be flawless	drop S7 Control Delegation first; [[ADR-0010-service-scope]]
Supabase free project pauses before the defence	dead demo	touch the project in the days before; local PostgreSQL container as the fallback path
GPS spoofing defeats the location licence	V.2.1 partially circumvented	event code as the stronger proof; residual risk stated as T-17 in [[threat-register]]
Single backend owner, no reviewer	a concurrency bug reaches the defence	parallel tests in the definition of done; prompt §4 per concurrency slice
YouTube quota exhausted during the demo	the demo fails in front of evaluators	caching and `videos.list` preference in [[youtube-integration]]; watch the quota counter
ADR written but not propagated	the Brain and the SDD contradict the code	step 4-6 of the closing loop; prompt §14
Copying from `reference/`	net6 packages pulled in, IV.1 violated	`reference/` is read-only; read the pattern, write your own

---

Progress tracking

Single source of truth for status: the traceability table in [[sdd]] §16.

Allowed statuses: `not started`, `designed`, `partial`, `done`. `done` requires code. Writing `done` for a designed-but-unwritten feature is the fastest way to fail a defence.

Open decisions: [[sdd]] §17. Close rows, never delete them.

Related

[[sdd]] · [[subject-requirements]] · [[decisions-index]] · [[owasp-index]] · [[user-manual]] · [[prompt-library]] · [[technical-guide-index]] · [[HOME]]